

多項式・形式的べき級数 線形代数 線形漸化的数列 C-recursive 行列累乗
Berlekamp-Massey のアルゴリズム Euclid の互除法 Half-GCD

線形漸化式の復元

1. 概要

本講座では、数列から定数係数線形漸化式を復元する問題について解説します。

例えば、Fibonacci 数列の先頭項

$$S = (0, 1, 1, 2, 3, 5, 8, 13)$$

を入力として、この数列が満たす漸化式

$$S_i = S_{i-1} + S_{i-2}$$

を出力するような問題を考えます。より正確には、有限数列が与えられたとき、その列と矛盾しない定数係数線形漸化式のうち、位数が最小のものを求める問題を扱います。この問題を、本講座では**復元問題**と呼びます。

復元問題を解く代表的なアルゴリズムが **Berlekamp-Massey アルゴリズム** です。

Berlekamp-Massey アルゴリズムは、入力列を先頭から順に処理し、各時点で数列が満たす最小位数の漸化式を更新していくアルゴリズムです。

競技プログラミングでは、Berlekamp-Massey アルゴリズムは、しばしば Bostan-Mori のアルゴリズム（または Fiduccia のアルゴリズム）と組み合わせて用いられます。具体的には、求めたい無限数列の先頭項を十分多く計算し、Berlekamp-Massey アルゴリズムで線形漸化式を復元し、Bostan-Mori のアルゴリズムで第 K 項を高速に求める、という流れです。この方法は **BMBM 法** と呼ばれることもあります。BMBM 法は考察や実装を簡略化できるだけでなく、計算量の改善につながることもあり、競技プログラミングでも重要な手法です。この方法を理解し実践できるようになることが、本講座の主目標となるでしょう。

6 節では補足として、Euclid の互除法を用いた復元問題の別解法を紹介します。このアルゴリズムは一見すると Berlekamp–Massey アルゴリズムとはかなり違う見ただけをしていますが、詳しく分析すると、Berlekamp–Massey アルゴリズムの計算と対応するものとして解釈することもできます。Euclid の互除法による見方は、Half-GCD による高速化につながるなどの利点もあります。

2. 前提知識

次の講座の内容を理解していることを前提とします。

- 線形漸化的数列
- 線形漸化的数列の第 K 項

また、加えて本講座で用いる多項式に関する用語についてここで簡単に補足します。

多項式 $f(x)$ の次数を $\deg f(x)$ と書きます。 $f(x) = 0$ の場合には $\deg 0 = -\infty$ と定義します。

多項式 $f(x)$ の最高次の係数が 1 であるとき $f(x)$ は**モニック**であるといいます。

多項式 $f(x), g(x)$ について少なくとも一方が 0 でないとき、 $f(x), g(x)$ を共に割り切る多項式であって、次数最大かつモニックであるものを $\gcd(f(x), g(x))$ と書きます。この多項式は通常の Euclid の互除法 と同様に計算できます。 $\gcd(f(x), g(x)) = 1$ であるような $f(x), g(x)$ は互いに素であるといいます。

3. 問題設定

これまでの講座

- 線形漸化的数列
- 線形漸化的数列の第 K 項

では、数列の要素や多項式の係数は、**単位的可換環** R の元であるとしてきました。

本講座では R は**体**であるとします。競技プログラミングでは、 $R = \mathbb{Q}$ （有理数全体）または $R = \mathbb{F}_p$ の場合（整数を素数 p を法として考える場合）を想定してよいでしょう。

また計算量について述べるときは、 R 上の四則演算の回数を数えることにします。

3.1. 線形漸化式の復元問題

本講座では、次の問題を解くことを目標とします。

漸化式による定式化

【問題 1】

長さ N の数列 $S = (S_0, S_1, \dots, S_{N-1})$ が与えられます。 S と矛盾しない定数係数線形漸化式であって、位数が最小のものをひとつ出力してください。

つまり、位数 L の定数係数線形漸化式

$$S_i = c_1 S_{i-1} + c_2 S_{i-2} + \dots + c_L S_{i-L}$$

が任意の $L \leq i < N$ について成り立つものであって、 L が最小のものをひとつ出力してください。

多項式による言い換え

線形漸化的数列と、有理式で表される形式的べき級数の対応を思い出せば、この問題は次のように言い換えることができます。

【問題 2】

$N - 1$ 次以下の多項式

$$S(x) = S_0 + S_1 x + \dots + S_{N-1} x^{N-1}$$

が与えられます。非負整数 L および L 次以下の多項式 $Q(x)$ (ただし $[x^0] Q(x) = 1$) であって、

$$[x^i] Q(x) S(x) = 0 \quad (L \leq i < N)$$

が成り立つものを考えます。そのような組 $(L, Q(x))$ のうち、 L が最小であるものをひとつ出力してください。

有理式による言い換え

条件

$$[x^i]Q(x)S(x) = 0 \quad (L \leq i < N)$$

は, $P(x) = Q(x)S(x) \bmod x^L$ を用いて

$$Q(x)S(x) \equiv P(x) \pmod{x^N}$$

または $[x^0]Q(x) = 1$ に注意して

$$S(x) \equiv \frac{P(x)}{Q(x)} \pmod{x^N}$$

と表すこともできます. したがって, **【問題 2】** は

- $[x^0]Q(x) = 1, \deg Q(x) \leq L,$
- $\deg P(x) \leq L - 1,$
- $S(x) \equiv \frac{P(x)}{Q(x)} \pmod{x^N}$

を満たすように $P(x), Q(x), L$ をとり, L を最小化する問題と言い換えることもできます. また L が最小である場合には

$$L = \max(\deg P(x) + 1, \deg Q(x))$$

が成り立ちます.

以下では, 本節で述べた問題を, どの定式化であるかを区別せずに, 単に**復元問題**と呼ぶことにします.

3.2. 入出力例

いくつかの例を通して, 想定されている入出力がどのようなものかを簡単に確認します. 先に本節で扱う入力列を挙げるので, 解説に入る前にどのような出力になるのか考えてみてもよいでしょう.

1. $S = (1, 2, 4, 8, 16, 32, 64)$
2. $S = (64, 32, 16, 8, 4, 2, 1)$

3. $S = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34)$
4. $S = (0, 1, 4, 9, 16, 25, 36, 49)$
5. $S = (10, 1, 1, 2, 3, 5, 8, 13, 21, 34)$
6. $S = (1, 0, 0, 0, 0)$
7. $S = (0, 0, 0, 0, 0)$
8. $S = (1, 2, 4, 7)$
9. $S = (0, 0, 0, 0, 1)$

なお、以下で解説する出力が復元問題の条件を満たすこと、特に L の最小性について、ここで完全に証明することはしません。また証明なしに、解の一意性について言及する場合もあります。これらについては、後述の **【定理 4】** により確かめることもできます。

本節の例において、係数環 R は $\mathbb{Q}$ （有理数全体）であるとします。

例 1 : $S = (1, 2, 4, 8, 16, 32, 64)$

公比が 2 の等比数列です。

- $L = 1$
- 漸化式 : $S_i = 2S_{i-1}$
- $Q(x) = 1 - 2x$

が正しい出力となります。なおこの場合、正しい出力は一意です。

例 2 : $S = (64, 32, 16, 8, 4, 2, 1)$

公比が $\frac{1}{2}$ の等比数列です。

- $L = 1$
- 漸化式 : $S_i = \frac{1}{2}S_{i-1}$
- $Q(x) = 1 - \frac{1}{2}x$

が正しい出力となります。なおこの場合、正しい出力は一意です。

このように、入力として与えられる数列が整数列でも、出力の係数が整数になるとは限りません.

例 3 : $S = (0, 1, 1, 2, 3, 5, 8, 13, 21, 34)$

Fibonacci 数列です.

- $L = 2$
- 漸化式 : $S_i = S_{i-1} + S_{i-2}$
- $Q(x) = 1 - x - x^2$

が正しい出力となります. なおこの場合, 正しい出力は一意です.

例 4 : $S = (0, 1, 4, 9, 16, 25, 36, 49)$

2 次多項式は線形漸化的数列の例であり, 母関数は $Q(x) = (1 - x)^3$ を分母とするような有理式表示を持ちます.

- $L = 3$
- 漸化式 : $S_i = 3S_{i-1} - 3S_{i-2} + S_{i-3}$
- $Q(x) = (1 - x)^3$

が正しい出力となります. なおこの場合, 正しい出力は一意です.

例 5 : $S = (10, 1, 1, 2, 3, 5, 8, 13, 21, 34)$

Fibonacci 数列から, S_0 のみが無関係な値に置き換わっています.

- $L = 3$
- 漸化式 : $S_i = S_{i-1} + S_{i-2} + 0S_{i-3}$
- $Q(x) = 1 - x - x^2 + 0x^3$

が正しい出力となります. なおこの場合, 正しい出力は一意です.

ここでは強調のため, 漸化式の項 $0S_{i-3}$ や $Q(x)$ の項 $0x^3$ を書いています.

このように, L と $Q(x)$ の次数が異なる場合もあることに注意してください.

出力はあくまでも、漸化式や $Q(x)$ そのものではなく、 L とそれらの組として扱う必要があります。

例 6 : $S = (1, 0, 0, 0, 0)$

- $L = 1$
- 漸化式 : $S_i = 0S_{i-1}$
- $Q(x) = 1 + 0x$

が条件を満たす出力となります。なおこの場合、正しい出力は一意です。

この例でも、 L と Q の次数は一致しません。

例 7 : $S = (0, 0, 0, 0, 0)$

- $L = 0$
- 漸化式 : $S_i = 0$ (右辺に項がひとつもない定数係数線形漸化式)
- $Q(x) = 1$

が条件を満たす出力となります。なおこの場合、正しい出力は一意です。

このように $L = 0$ となるのは S の要素がすべて 0 である場合に限られます。

例 8 : $S = (1, 2, 4, 7)$

L が最小であるような出力が複数あります。例えば

- $L = 3$
- 漸化式 : $S_i = S_{i-1} + S_{i-2} + S_{i-3}$
- $Q(x) = 1 - x - x^2 - x^3$

が条件を満たす出力となります。他に例えば

- $L = 3$
- 漸化式 : $S_i = 7S_{i-3}$
- $Q(x) = 1 - 7x^3$

が条件を満たす出力となります。

この例において $L = 2$ が不可能であることは、連立方程式

$$4 = 2c_1 + 1c_2, \quad 7 = 4c_1 + 2c_2$$

が解を持たないことから分かります。 $L = 3$ が複数の解を持つことは、方程式

$$7 = 4c_1 + 2c_2 + 1c_3$$

が複数の解を持つことから分かります。

例 9 : $S = (0, 0, 0, 0, 1)$

$L = N = 5$ が条件を満たす最小の L となります。また、 $L = 5$ を満たすような任意の定数係数線形漸化式が、正しい出力となります。

実際、成り立つべき条件は、 $L \leq i < N$ であるようなすべての i について何らかの等式が成り立つというもので、 $L = N$ の場合には自動的に条件が満たされます。

また、 $S_4 = 1$ を S_0, S_1, S_2, S_3 の線形結合で表すのは不可能なので、 $L \leq 4$ とした場合には条件を満たすことはできません。

このように $L = N$ は常に最小性を除き問題の条件を満たすため、問題の解は常に存在します。

入出力例に関する注意

例 9 で見たように $L = N$ とすると、任意の定数係数線形漸化式、あるいは任意の L 次以下の多項式 $Q(x)$ (ただし $[x^0]Q(x) = 1$) が復元問題の条件のうち L の最小性以外の部分をすべて満たします。したがって、復元問題の解は少なくともひとつ存在することが分かります。

また、例 8, 例 9 で見たように、 L が最小であるような解が複数存在する場合があります。解の一意性については 3.3 節で扱います。

例 5, 例 6 で見たように、 L と $Q(x)$ の次数は一致するとは限らないことにも注意が必要です。

3.3. 解の一意性について

例 8, 例 9 で見たように, 復元問題の解は一意に定まるとは限りません.

どのような場合に解の一意性が成り立たず, どのような場合に解の一意性が成り立つのかについて確認しておきます.

【定理 3】

長さ N の数列 $S = (S_0, S_1, \dots, S_{N-1})$ に対して, 復元問題の解のひとつを $(L, Q(x))$ とする. このとき, $N < 2L$ ならば, 解は複数存在する.

L を固定して, $Q(x) = 1 - \sum_{i=1}^L c_i x^i$ とします. このとき, $Q(x)$ が満たすべき条件

$$[x^i] Q(x)S(x) = 0 \quad (L \leq i < N)$$

は, 未知数の個数が $c_1, c_2, \dots, c_L$ の L 個, 方程式の個数が $L \leq i < N$ に対応する $N - L$ 個の連立 1 次方程式と見なすことができます.

未知数の個数が方程式の個数よりも多い連立 1 次方程式は, 解を持つならば複数の解を持つので, $N < 2L$ ならば複数の解が存在すると分かります. ■

【定理 4】

長さ N の数列 $S = (S_0, S_1, \dots, S_{N-1})$ に対して, 復元問題の解のひとつを $(L, Q(x))$ とする. このとき, $2L \leq N$ ならば, 解は唯一である.

$P(x) = Q(x)S(x) \bmod x^L$ とすれば, P は $L - 1$ 次以下の多項式で,

$$\frac{P(x)}{Q(x)} = S(x) \pmod{x^N}$$

が成り立つのでした. まず, $L = \max(\deg P(x) + 1, \deg Q(x))$ の最小性より, $P(x), Q(x)$ は互いに素であることが分かります. そうでないとすると, $\gcd(P(x), Q(x))$ を定数項が 1 になるように定数倍したものを $g(x)$ とし,

$$P_1(x) = P(x)/g(x), \quad Q_1(x) = Q(x)/g(x), \quad L_1 = L - \deg g(x)$$

とすれば, $\frac{P_1(x)}{Q_1(x)} = \frac{P(x)}{Q(x)} \equiv S(x) \pmod{x^N}$ より $(L_1, Q_1(x))$ は $L_1 < L$ であるような解になり, L の最小性に矛盾するからです.

次に $(L, Q_1(x))$ も解であるとし, $P_1(x) = Q_1(x)S(x) \pmod{x^L}$ とします. すると

$$\frac{P(x)}{Q(x)} \equiv S(x) \equiv \frac{P_1(x)}{Q_1(x)} \pmod{x^N}$$

より

$$P(x)Q_1(x) \equiv Q(x)P_1(x) \pmod{x^N}$$

が成り立ちます.

$$\deg P(x) \leq L-1, \quad \deg Q(x) \leq L, \quad \deg P_1(x) \leq L-1, \quad \deg Q_1(x) \leq L$$

より $P(x)Q_1(x), Q(x)P_1(x)$ はともに $2L-1$ 次以下です. $2L \leq N$ よりこれらは $N-1$ 次以下です. したがってこの「 x^N を法とする合同」は実際には等号で,

$$P(x)Q_1(x) = Q(x)P_1(x), \quad \frac{P(x)}{Q(x)} = \frac{P_1(x)}{Q_1(x)}$$

が成り立つことが分かります. $P(x), Q(x)$ が互いに素であることから, このようなことが成り立つのはある多項式 $g(x)$ に対して

$$P_1(x) = P(x)g(x), \quad Q_1(x) = Q(x)g(x)$$

となる場合に限られます.

$$L = \max(\deg P(x) + 1, \deg Q(x)) = \max(\deg P_1(x) + 1, \deg Q_1(x))$$

なので, $\deg g(x) = 0$ となるしかありません. これと $[x^0]Q(x) = [x^0]Q_1(x) = 1$ より $Q(x) = Q_1(x)$ となるので, 解の一意性が示されました. ■

4. Berlekamp-Massey アルゴリズム

本節では, 復元問題を解く代表的なアルゴリズムである Berlekamp-Massey アルゴリズムを説明します.

4.1. 全体の流れ

Berlekamp–Massey アルゴリズムは、入力列

$$S = (S_0, S_1, \dots, S_{N-1})$$

を先頭から順に処理します。 $n = 0, 1, \dots, N - 1$ の順に処理し、 n 番目のイテレーションの終了時点で、

$$S_0, S_1, \dots, S_n$$

に対する復元問題の解 $(L, Q(x))$ を求めている、というのが目標です。

n 番目のイテレーションの開始時点で、すでに

$$S_0, S_1, \dots, S_{n-1}$$

に対する解 $(L, Q(x))$ が得られているとします。 まずこの $Q(x)$ について、

$$\Delta = [x^n] S(x) Q(x)$$

を計算します。 $\Delta = 0$ ならば、現在の $Q(x)$ は S_n についても条件を満たしているので、何も更新する必要はありません。

一方、 $\Delta \neq 0$ の場合には、現在の $Q(x)$ では S_n に対する条件を満たしません。 そこで、 $Q(x)$ に適切な多項式を足すことで、この Δ を打ち消します。（ただし、初回の更新だけは例外で、 L を大きくすることで係数条件を自動的に満たすようにします。）

このとき、「前回 L を増加させたときの情報」を保存しておき、それを現在位置までシフトして使う、というのが Berlekamp–Massey アルゴリズムのアイデアです。 次節の疑似コードでは、そのときの

- $Q(x)$ を $B(x)$ という変数、
- n を n_0 という変数、
- Δ を b という変数、

で保持しています。

4.2. 疑似コード

Berlekamp–Massey の疑似コードは次の通りです。

```

Berlekamp_Massey(S):
  N := length(S)

  Q(x) := 1
  L := 0

  B(x) := 1
  n0 := -1
  b := 1

  for n = 0, 1, ..., N-1:
    Delta := [x^n] Q(x)S(x)
    if Delta = 0:
      continue

    Q_new(x) := Q(x) - (Delta / b) x^(n-n0) B(x)

    if 2L <= n:
      L_new := n + 1 - L
      B(x) := Q(x)
      n0 := n
      b := Delta
    else:
      L_new := L

    Q(x) := Q_new(x)
    L := L_new

  return (L, Q(x))

```

初期値

$$B(x) = 1, \quad n_0 = -1, \quad b = 1$$

はやや特殊ですが、最初に $\Delta \neq 0$ となったときの更新を他の場合と同じ形で書くための番兵です。

この疑似コードが復元問題を正しく解くことを、4.3, 4.4 節で確認します。

4.3. 正当性：係数に関する条件

まず、各イテレーションの終了時点で、 $(L, Q(x))$ が係数に関する条件

$$[x^i]S(x)Q(x) = 0 \quad (L \leq i \leq n)$$

を満たしていることを確認します。ここでは L の最小性は扱わず、最小性は次節で証明します。

n 番目のイテレーションの開始時点で、

$$[x^i]S(x)Q(x) = 0 \quad (L \leq i < n)$$

が成り立っているとします。まず $\Delta = [x^n]S(x)Q(x)$ を計算します。 $\Delta = 0$ ならば、現在の $Q(x)$ は n 番目の係数についても条件を満たしているので、何も更新せずとも $(L, Q(x))$ は先頭 $n+1$ 項に対する条件を満たします。

以下、 $\Delta \neq 0$ の場合を考えます。

$n_0 = -1$ の場合

まず、 $n_0 = -1$ の場合を考えます。これは、これまで一度も L を増加させておらず、 $Q(x) = 1, L = 0$ となっています。更新式は

$$Q_{\text{new}}(x) = 1 - \Delta x^{n+1}, \quad L_{\text{new}} = n + 1$$

となります。 $L_{\text{new}} \leq i \leq n$ を満たす i は存在しないので、条件

$$[x^i]S(x)Q_{\text{new}}(x) = 0 \quad (L_{\text{new}} \leq i \leq n)$$

は自動的に満たされます。

$n_0 \geq 0$ の場合

以下では $n_0 \geq 0$ とします。この場合、この時点での $B(x), n_0, b$ について

$$b = [x^{n_0}]S(x)B(x)$$

が成り立っています。 $Q(x)$ の更新式は

$$Q_{\text{new}}(x) = Q(x) - \frac{\Delta}{b} x^{n-n_0} B(x)$$

です。まず、この更新によって n 番目の係数が 0 になることが次の計算により確認できます。

$$\begin{aligned} [x^n] Q_{\text{new}}(x) S(x) &= [x^n] Q(x) S(x) - \frac{\Delta}{b} \cdot [x^n] x^{n-n_0} B(x) S(x) \\ &= \Delta - \frac{\Delta}{b} \cdot b = 0 \end{aligned}$$

次に、 $L_{\text{new}} \leq i < n$ 次の係数に関する条件を確認します。

$B(x)$ が保存された n_0 番目のイテレーションにおける様子を考えます。このイテレーション開始時点での位数を L_0 と書くことにすると、

- $[x^i] B(x) S(x) = 0 \quad (L_0 \leq i < n_0)$
- 現在の L について $L = n_0 + 1 - L_0$

が成り立つことが分かります。前者の式の x^{n-n_0} 倍から

$$[x^i] x^{n-n_0} B(x) S(x) = 0 \quad (L_0 + n - n_0 \leq i < n)$$

が成り立ちます。ここで $L = n_0 + 1 - L_0$ より $L_0 + n - n_0 = n + 1 - L$ です。さらに、更新式における $2L \leq n, n < 2L$ のどちらの分岐の場合でも $n + 1 - L \leq L_{\text{new}}$ が成り立ちます。したがって

$$[x^i] x^{n-n_0} B(x) S(x) = 0 \quad (L_{\text{new}} \leq i < n)$$

が成り立ちます。このことから更新後の $(L_{\text{new}}, Q_{\text{new}}(x))$ が

$$[x^i] Q_{\text{new}}(x) S(x) = 0 \quad (L_{\text{new}} \leq i \leq n)$$

を満たすことが分かります。

また、 $\deg Q_{\text{new}}(x) \leq L_{\text{new}}$ も成り立つことを確認しておきます。 $\deg B(x) \leq L_0$ と $L = n_0 + 1 - L_0$ から、

$$\deg x^{n-n_0} B(x) \leq (n - n_0) + L_0 = n + 1 - L$$

で、これは L_{new} 以下です。したがって $\deg Q_{\text{new}}(x) \leq L_{\text{new}}$ が成り立ちます。

4.4. 正当性： L の最小性

前節では、各イテレーションの終了時点で、 $(L, Q(x))$ が係数に関する条件を満たしていることを確認しました。本節では、その L が最小であることを確認します。

n 番目のイテレーションの開始時点で、 $(L, Q(x))$ は

$$S_0, S_1, \dots, S_{n-1}$$

に対して係数に関する条件を満たしており、さらに L は最小であると仮定します。

まず $\Delta = 0$ の場合を考えます。この場合、現在の $(L, Q(x))$ はそのまま先頭 $n + 1$ 項に対する係数に関する条件を満たします。もし、先頭 $n + 1$ 項に対して L より小さい位数の解が存在したならば、それは先頭 n 項に対する解にもなります。これは開始時点での L の最小性に反します。したがって、 $\Delta = 0$ の場合には L は最小のままです。

次に $\Delta \neq 0$ の場合を考えます。

まず $2L > n$ の場合です。この場合、疑似コードでは $L_{\text{new}} = L$ とします。この場合もやはり、 L より小さい位数の解は、先頭 n 項に対する解にもなってしまいうので存在しません。したがって、 $L_{\text{new}} = L$ は最小です。

残るのは、 $\Delta \neq 0$ かつ $2L \leq n$ の場合です。この場合、疑似コードでは

$$L_{\text{new}} = n + 1 - L$$

とします。この値が必要最小であることを示します。 $(L, Q(x))$ は先頭 n 項に対する復元問題の解なので、 $P(x) = Q(x)S(x) \bmod x^L$ とおけば、

- $\deg Q(x) \leq L$
- $\deg P(x) \leq L - 1$
- $S(x) \equiv \frac{P(x)}{Q(x)} \pmod{x^n}$

が成り立つのでした。 $(L_1, Q_1(x))$ を先頭 $n + 1$ 項に対する復元問題の解として、 $L_1 \leq n - L$ が成り立つと仮定します。 $P_1(x) = Q_1(x)S(x) \bmod x^{L_1}$ とすると

- $\deg Q_1(x) \leq L_1$
- $\deg P_1(x) \leq L_1 - 1$
- $S(x) \equiv \frac{P_1(x)}{Q_1(x)} \pmod{x^{n+1}}$

が成り立ちます。したがって特に $\frac{P(x)}{Q(x)} \equiv \frac{P_1(x)}{Q_1(x)} \pmod{x^n}$ が成り立ちます。よって

$$P(x)Q_1(x) \equiv P_1(x)Q(x) \pmod{x^n}$$

です。この両辺の次数は $L + L_1 - 1$ 以下ですが、 $L_1 \leq n - L$ の仮定から $n - 1$ 次以下です。したがってこの「 x^n を法とする合同」は実際には等号で、

$$P(x)Q_1(x) = P_1(x)Q(x) \quad \frac{P(x)}{Q(x)} = \frac{P_1(x)}{Q_1(x)}$$

が成り立つことが分かります。一方、 $(L, Q(x))$ は先頭 $n + 1$ 項に対する復元問題の解ではなかったので、

$$\frac{P_1(x)}{Q_1(x)} = \frac{P(x)}{Q(x)} \not\equiv S(x) \pmod{x^{n+1}}$$

です。これは $(L_1, Q_1(x))$ が先頭 $n + 1$ 項に対する復元問題の解であることに矛盾します。

以上で、 $L_1 \leq n - L$ が成り立つと仮定して矛盾が生じたため、先頭 $n + 1$ 項に対する復元問題の解は $L_1 \geq n + 1 - L$ を満たすことが示されました。4.3 節で確認したように、アルゴリズムは実際に $L_{\text{new}} = n + 1 - L$ となる解を構成するため、アルゴリズムの L は最小です。

4.5. 計算量

係数環 R における四則演算を $O(1)$ とする場合、Berlekamp–Massey アルゴリズムは $O(N^2)$ 時間で復元問題の解を出力します。

4.6. 実装例

[Library Checker "Find Linear Recurrence"](#) への提出です。

- <https://judge.yosupo.jp/submission/379599>

5. 競技プログラミングでの利用

本節では、競技プログラミングにおける Berlekamp–Massey アルゴリズムの典型的な利用方法について解説します。

5.1. BMBM 法

ここまででは、長さ N の有限列が与えられたときに、それと矛盾しない最小位数の定数係数線形漸化式を復元する問題を扱いました。一方、競技プログラミングでの典型的な使い方は、有限列そのものに興味があるというよりも、背後にある無限数列を推定し、その第 K 項を高速に求めるというものです。

典型的には、次のような流れとなります。

1. 求めたい無限数列 $A_0, A_1, A_2, \dots$ について、適当なしきい値 N を設定し、先頭 N 項を何らかの方法で計算する。
2. Berlekamp–Massey アルゴリズムで、その先頭 N 項から漸化式を復元する。
3. 復元された漸化式を用いて、第 K 項を高速に計算する。

第 K 項計算には、前講座で扱った Bostan–Mori のアルゴリズムなどを用いることができます（参考：[線形漸化的数列の第 \$K\$ 項](#)）。この

Berlekamp–Massey $\longrightarrow$ Bostan–Mori

という一連の流れを、**BMBM 法** と呼ぶこともあります。

まず、考えている無限数列が位数 L 以下の定数係数線形漸化式を満たす線形漸化的数列であることが分かっている状況を考えます。3.3 節で見たように、 $2L \leq N$ が成り立つ場合には、復元問題の解は一意に定まります。したがって、 $2L \leq N$ となるように N を設定し、先頭 N 項を計算して Berlekamp–Massey アルゴリズムに渡すことで、その無限数列が満たす定数係数線形漸化式のうち最小位数のものを正しく復元することができます。

一方で、考えている無限数列について何の手がかりがない場合でも、 N を十分大きくとり Berlekamp–Massey アルゴリズムを用いると、何らかの位数 L の定数係数線形漸化式が出力されます。ただしこの場合の出力は、先頭 N 項に対して矛盾しない漸化式であるというだけで、元の無限数列がその漸化式を満たすことも、線形漸化的数列であることも意味しません。したがってアルゴリズムが何らかの出力を返しても、無限数列の満たす定数係数線形漸化式が復元できたと考えるのは不適切です。

ただし、出力された位数 L について $2L$ が N に比べて十分小さい場合には、得られた漸化式にはある程度の信憑性があると考えてもよいでしょう。

適切にしきい値 N を設定したり、ある出力の妥当性を検証するためには、対象の数列为線形漸化的であることや、その位数の上界を別途考えられることが望ましいです。この点については、線形漸化的数列 で学んだ内容が大いに役に立つと思います。

5.2. 例題

次の問題は、手計算ではなくプログラムによる計算で答えを求めることが想定されています。実際の競技プログラミングの出題ではなく、解説のための人工的な問題です。

複雑に見える数列に対しても、先頭項を素直に計算して BMBM 法を適用するだけで答えを求められることを確認します。特に、実際に行列を構成したり、最小位数の漸化式を手で導出したりする必要がない点に注目してください。

【問題 5】

非負整数列 $A = (A_0, A_1, A_2, \dots)$, $B = (B_0, B_1, B_2, \dots)$, $C = (C_0, C_1, C_2, \dots)$ が以下を満たすとします。

$$\begin{aligned} A_0 &= 1, & B_0 &= 1, & C_0 &= 1, \\ A_i &= B_{i-1} + 2C_{i-1} + 2^{i-1} & (1 \leq i), \\ B_i &= 3A_{i-1} + 4C_{i-1} + (i-1)^2 & (1 \leq i), \\ C_i &= 5A_{i-1} + 6B_{i-1} + F_{i-1} & (1 \leq i). \end{aligned}$$

ここで $F = (F_0, F_1, F_2, F_3, \dots) = (0, 1, 1, 2, \dots)$ は Fibonacci 数列です。さらに、 $K = 10^{18}$ とします。

1. $A_K \bmod 998244353$ を求めてください。
2. $\left(\sum_{i=0}^K A_i\right) \bmod 998244353$ を求めてください。
3. $\left(\sum_{i=0}^K A_i B_i C_i\right) \bmod 998244353$ を求めてください。

▶ 答えの値

この解法では、実際に行列を構成したり、 $A_i B_i C_i$ を状態に含めるための遷移を設計したりする必要がありません。先頭項を素直に計算し、得られた列に BMBM 法を適用するだけでよいので、（ある程度ライブラリをそろえている場合には）実装が非常に簡潔になります。

一方でこの解法は、実装を簡略化するためだけのものではなく、計算量を改善するという観点からも重要なものです。このことを次節で紹介します。

5.3. 行列累乗への応用

M を $d \times d$ 行列とし、ある問題の答え A_i が行ベクトル u^T と列ベクトル v を用いて

$$A_i = u^T M^i v$$

と表せるとしましょう。例えば M^i のひとつの成分が答えである場合や、ひとつの列の成分の線形結合である場合がこれに相当します。第 K 項 A_K を求める問題について考えましょう。

解法 1：行列累乗による方法

まずこの問題は、行列累乗を繰り返し二乗法により計算することで解くことができます。素朴な行列積の計算アルゴリズムを用いる場合、計算量は

$$O(d^3 \log K)$$

です。

解法 2：BMBM 法

次に、この問題を BMBM 法で解くことを考えてみましょう。Cayley–Hamilton の定理より、 A_i は位数 d 以下の定数係数線形漸化式を満たす線形漸化的数列です。したがって、 A の先頭 $2d$ 項を求めてから、BMBM 法によって答えを求めるという計算手順が考えられます。この方法の計算量について考えます。

小さな i について A_i を求める必要がありますが、この際行列 M^i そのものを求める必要はありません。具体的には、列ベクトル

$$V_i := M^i v$$

を求めたあと、 $A_i = u^T V_i$ により答えを取り出せばよいです。 V_i の計算には漸化式

$$V_{i+1} = M V_i$$

を用いることができます。この計算は（行列と行列の積ではなく）行列と列ベクトルの積の形なので、1 回あたり $O(d^2)$ 時間で行えます。結局、 A の先頭 $2d$ 項の計算は $O(d^3)$ 時間で行えることが分かります。

先頭項からの漸化式の復元は Berlekamp–Massey アルゴリズムで $O(d^2)$ 時間で行うことができ、さらに第 K 項の計算は Bostan–Mori のアルゴリズムで $O(M(d) \log K)$ 時間で行うことができます（ $M(d)$ は d 次多項式の積を求める計算量）。全体をまとめると、第 K 項 A_K は

$$O(d^3 + M(d) \log K)$$

時間で求めることができます。特に d, K がある程度大きい場合には、（多項式乗算を素朴な方法で行ったとしても）繰り返し二乗法による行列累乗よりも高速に答えを求められていることが確認できます。

行列が疎な場合

行列 M が疎である場合、つまり行列の成分のうち、0 でないものが少ない場合には、計算量の観点でさらに BMBM 法が有利となります。

例えば、 $d \times d$ 行列 M の成分のうち、0 でないものの個数が $O(d)$ 個である場合には、BMBM 法の計算量は

$$O(d^2 + M(d) \log K)$$

となることが上の議論から分かります。行列と列ベクトルの積 $M V_i$ を求めることが $O(d)$ 時間で行えるためです。

6. 参考：Euclid の互除法による復元

本節では、Berlekamp–Massey アルゴリズムとは別の方法として、多項式の拡張 Euclid の互除法を利用した復元問題の解法を紹介します。

6.1. 係数順を逆にした復元問題

まず復元問題に現れる多項式 $S(x)$, $Q(x)$ の係数の並び順を逆順に変換して定式化しなおします.

$$S = (S_0, S_1, \dots, S_{N-1})$$

に対して, $N - 1$ 次までの係数の並び順を逆順にした多項式

$$T(x) = x^{N-1} S(x^{-1}) = S_{N-1} + S_{N-2}x + \dots + S_0 x^{N-1}$$

を考えます ($N = 0$ の場合は $T(x) = 0$ とします). 同様に, 復元問題の解を $(L, Q(x))$ とするとき, $Q(x)$ の L 次までの係数の並び順を逆順にした多項式

$$\tilde{Q}(x) = x^L Q(x^{-1})$$

を考えます. $[x^0]Q(x) = 1$ から, $\tilde{Q}(x)$ はモニックな L 次多項式です. 次に係数に関する条件

$$[x^i]Q(x)S(x) = 0 \quad (L \leq i < N)$$

を書き直しましょう. $Q(x)S(x)$ と $\tilde{Q}(x)T(x)$ にも, 係数の並び順を逆順にしたという関係があります. より正確には

$$\tilde{Q}(x)T(x) = x^{N+L-1} Q(x^{-1})S(x^{-1})$$

です. したがって, $Q(x)S(x)$ の L 次以上 $N - 1$ 次以下の係数が 0 であることは, $\tilde{Q}(x)T(x)$ の L 次以上 $N - 1$ 次以下の係数が 0 であることと同値です. つまり, ある多項式 $\tilde{P}(x)$ が存在して

$$\tilde{Q}(x)T(x) \equiv \tilde{P}(x) \pmod{x^N}, \quad \deg \tilde{P}(x) < L = \deg \tilde{Q}(x)$$

が成り立つことと同値です. したがって, 復元問題は次のように言い換えられます.

【問題 6】

$T(x)$ に対して, モニック多項式 $\tilde{Q}(x)$ および多項式 $\tilde{P}(x)$ の組であって,

$$\tilde{Q}(x)T(x) \equiv \tilde{P}(x) \pmod{x^N}, \quad \deg \tilde{P}(x) < \deg \tilde{Q}(x)$$

を満たすもののうち, $\deg \tilde{Q}(x)$ が最小のものを求めてください.

6.2. アルゴリズム

【問題 6】 は、 x^N と $T(x)$ に対する拡張 Euclid の互除法で解くことができます。

まず、 x^N と $T(x)$ に対する Euclid の互除法を最後まで行ったときに得られる剰余列

$$r_{-1}(x), r_0(x), r_1(x), \dots$$

を考えます。初期値を

$$r_{-1}(x) = x^N, \quad r_0(x) = T(x)$$

とし、 $r_i(x) \neq 0$ である限り、商 $q_i(x)$ と剰余 $r_{i+1}(x)$ を

$$r_{i+1}(x) = r_{i-1}(x) - q_i(x)r_i(x), \quad \deg r_{i+1}(x) < \deg r_i(x)$$

で定めます。ただし、ある時点で $r_i(x) = 0$ となった場合には、それ以降の剰余は定義しません。拡張 Euclid の互除法ではさらに、各剰余 $r_i(x)$ を $T(x)$ と x^N の線形結合として表す係数も同時に管理します。すなわち、多項式 $u_i(x), v_i(x)$ を用いて

$$r_i(x) = u_i(x)T(x) + v_i(x)x^N$$

となるようにします。具体的な初期値および更新式は

$$\begin{aligned} u_{-1}(x) &= 0, & v_{-1}(x) &= 1, & u_0(x) &= 1, & v_0(x) &= 0, \\ u_{i+1}(x) &= u_{i-1}(x) - q_i(x)u_i(x), \\ v_{i+1}(x) &= v_{i-1}(x) - q_i(x)v_i(x) \end{aligned}$$

です。特に、 x^N を法として見れば、常に

$$u_i(x)T(x) \equiv r_i(x) \pmod{x^N}$$

が成り立ちます。この式が、【問題 6】 の条件 $\tilde{Q}(x)T(x) \equiv \tilde{P}(x) \pmod{x^N}$ と同じ形であることに注意しましょう。

Euclid の互除法を進めると、 $r_i(x)$ の次数は狭義単調減少し、 $u_i(x)$ の次数は狭義単調増加します。復元アルゴリズムでは、剰余列をはじめて

$$\deg r_i(x) < \deg u_i(x)$$

となるところまで計算し，出力をモニツクにするための定数倍をした上でそれを出します．つまり， $\deg r_i(x) < \deg u_i(x)$ を満たす最小の i を $i = k$ とし，

$$L = \deg u_k(x), \quad c = [x^L]u_k(x)$$

としたとき

$$\tilde{Q}(x) = \frac{u_k(x)}{c}, \quad \tilde{P}(x) = \frac{r_k(x)}{c}$$

を出力とするのが 【問題 6】 を解くアルゴリズムとなります．

このような k は必ず存在します．実際，Euclid の互除法を最後まで進めれば，いずれ $r_i(x) = 0$ になり，停止条件 $\deg r_i(x) < \deg u_i(x)$ を満たします．このアルゴリズムの正当性，つまり出力について $\deg \tilde{Q}(x)$ の最小性が成り立つことは 6.3 節で証明します．

上で述べたアルゴリズムを疑似コードにすると次のようになります．なお結論を見ると分かるように，係数 $u_i(x), v_i(x)$ のうちで出力に必要なのは $u_i(x)$ だけです．そこで以下の疑似コードでは $v_i(x)$ は管理せず， $r_i(x)$ と $u_i(x)$ だけを管理します．また，次の疑似コードでは，最後に $\tilde{Q}(x)$ の係数の並び順を逆順にすることで，元の復元問題の解 $(L, Q(x))$ を出力としています．

```
Euclid_Reconstruct(S):
  N := length(S)

  T(x) := S_{N-1} + S_{N-2} x + ... + S_0 x^{N-1}

  r_prev(x) := x^N
  r(x) := T(x)
  u_prev(x) := 0
  u(x) := 1

  while deg r >= deg u:
    q(x), r_next(x) := quotient and remainder of r_prev(x) divided by r(x)

    u_next(x) := u_prev(x) - q(x) u(x)
    r_prev(x) := r(x)
    r(x) := r_next(x)
    u_prev(x) := u(x)
    u(x) := u_next(x)
```

```

L := deg u(x)
c := [x^L] u(x)
u(x) := u(x) / c

Q(x) := x^L u(x^{-1})
return (L, Q(x))

```

6.3. 正当性の証明

6.2 節のアルゴリズムが **【問題 6】** の解を返すことを示します。6.2 節で見たように、停止した時点の $u_k(x), r_k(x)$ は

$$u_k(x) T(x) \equiv r_k(x) \pmod{x^N}, \quad \deg r_k(x) < \deg u_k(x)$$

を満たします。したがって、定数倍してモニックにした出力は **【問題 6】** の条件を満たします。以下では、その次数が最小であることを示します。

まず剰余列 $r_i(x)$ と係数 $u_i(x)$ について、

$$\deg u_i(x) = N - \deg r_{i-1}(x)$$

が成り立ちます。このことは帰納法により簡単に示すことができます。

$u_k(x)$ よりも低次数の解が存在しないことを示します。 $L = \deg u_k(x)$ とおきます。 $\deg U(x) < L$ である 0 でない多項式 $U(x)$ と多項式 $R(x)$ について

$$U(x) T(x) \equiv R(x) \pmod{x^N}, \quad \deg R(x) < \deg U(x)$$

が成り立つと仮定します。合同式

$$\begin{aligned} u_{k-1}(x) T(x) &\equiv r_{k-1}(x) \pmod{x^N}, \\ U(x) T(x) &\equiv R(x) \pmod{x^N} \end{aligned}$$

より、

$$R(x) u_{k-1}(x) \equiv u_{k-1}(x) U(x) T(x) \equiv r_{k-1}(x) U(x) \pmod{x^N}$$

つまり

$$R(x)u_{k-1}(x) - r_{k-1}(x)U(x) \equiv 0 \pmod{x^N}$$

が成り立ちます。ここでアルゴリズムが $i = k - 1$ で停止しなかったことより $\deg u_{k-1}(x) \leq \deg r_{k-1}(x)$ です（これは $k = 0$ の場合にも成り立ちます）。これと $R(x), U(x)$ に関する仮定 $\deg R(x) < \deg U(x)$ から

$$\deg(R(x)u_{k-1}(x)) < \deg(r_{k-1}(x)U(x))$$

が成り立ちます。したがって非負整数 d を $d = \deg r_{k-1}(x) + \deg U(x)$ により定めると、 $R(x)u_{k-1}(x) - r_{k-1}(x)U(x)$ は d 次に 0 でない係数を持ちます。一方上で示した $\deg u_k(x) = N - \deg r_{k-1}(x)$ と $\deg U(x) < \deg u_k(x)$ から

$$d = \deg(r_{k-1}(x)U(x)) = \deg r_{k-1}(x) + \deg U(x) = N - \deg u_k(x) + \deg U(x) < N$$

です。これは $R(x)u_{k-1}(x) - r_{k-1}(x)U(x) \equiv 0 \pmod{x^N}$ に矛盾します。

よって、 $\deg U(x) < \deg u_k(x)$ で **【問題 6】** の条件を満たす 0 でない多項式 $U(x)$ は存在しないため、6.2 節のアルゴリズムの正当性が示されました。

6.4. 計算量

係数環 R における四則演算を $O(1)$ とする場合、6.2 節のアルゴリズムは $O(N^2)$ 時間で復元問題の解を出力します。

6.5. Berlekamp-Massey アルゴリズムとの関係

6.2 節の Euclid の互除法によるアルゴリズムと Berlekamp-Massey アルゴリズムは、どちらも同じ復元問題を解いています。

これらの計算過程にも対応があります。具体的には、Euclid の互除法の多項式除算を、より細かく、商多項式の各項を使って先頭係数をひとつずつ消す操作に細分すると、（係数の並び順の反転や定数倍を適切に対応させれば）Berlekamp-Massey アルゴリズムと対応することが知られています。詳しくは、[文献 4](#)、[文献 5](#) を参照してください。

なお、最小位数 L が $2L \leq N$ を満たす場合には、両アルゴリズムの出力は一致します。このことは復元問題の解の一意性からも明らかです。

一方で $N < 2L$ の場合には、通常の Euclid の互除法と Berlekamp–Massey アルゴリズムの終了条件が完全には対応しないため、両アルゴリズムの出力は一致するとは限りません。

6.6. Half-GCD による高速化

Half-GCD と呼ばれるアルゴリズムを用いると、計算量オーダーを改善することが可能です。

Half-GCD とは、多項式の Euclid の互除法について、複数ステップをまとめて計算するアルゴリズムです。

通常の Euclid の互除法では、剰余の組 $(r_{i-1}(x), r_i(x))$ から 1 ステップずつ剰余の組 $(r_i(x), r_{i+1}(x))$ へ進みます。一方で Half-GCD では、この複数ステップ

$$(r_{i-1}(x), r_i(x)) \rightarrow (r_{j-1}(x), r_j(x))$$

をまとめて計算します。正確な仕様は実装依存がありますが、例えばはじめて $\deg r_j(x) \leq \frac{1}{2} \deg r_{i-1}(x)$ となるところまでを計算します。

6.2 節の復元問題では、Euclid の互除法を

$$\deg r_i(x) < \deg u_i(x)$$

となるところまで進めるのでした。 $\deg u_i(x) = N - \deg r_{i-1}(x)$ なので、この条件は、隣り合う剰余の次数だけを用いて

$$\deg r_{i-1}(x) + \deg r_i(x) < N$$

と表せます。 $\deg r_i(x)$ が狭義単調減少することから、この条件がはじめて成り立つのは、はじめて $\deg r_i(x) \leq N/2$ になるような i またはその次の i です。したがって、Half-GCD を実行したあと、通常の互除法を数ステップ進めることで、ちょうど復元問題の終了条件まで Euclid の互除法を進めることができます。

Half-GCD は $O(M(N) \log N)$ 時間で計算できるため、復元問題を $O(M(N) \log N)$ 時間で解くことができます。

本講座では Half-GCD アルゴリズムの解説はしませんが、今後の講座で改めて解説します。その際に改めて本講座の復元問題への利用についても確認する予定です。

- 実装例：<https://judge.yosupo.jp/submission/379831>

ただし、競技プログラミングでの出題例として、一般の復元問題を $O(M(N) \log N)$ 時間で解くアルゴリズムが想定された問題は、筆者の知る限りは存在しません。BMBM 法を用いる場合にも、先頭 $N = 2L$ 項の計算に $O(N^2)$ 時間程度かかることが多く、Berlekamp–Massey アルゴリズムが問題を解く際のボトルネックになりにくいのです。そのため、本講座の内容を競技プログラミングで実用する上では、通常は $O(N^2)$ 時間で復元問題が解ければ十分だと思います。

6.7. Padé 近似との関係

形式的べき級数 $f(x)$ ，非負整数 a, b に対して，多項式 $P(x), Q(x)$ であって

$$\begin{aligned} \deg P(x) &\leq a, & \deg Q(x) &\leq b, & [x^0] Q(x) &= 1, \\ f(x) &\equiv \frac{P(x)}{Q(x)} \pmod{x^{a+b+1}} \end{aligned}$$

を満たすものを， $f(x)$ の (a, b) **Padé 近似**と呼びます。

この条件を満たす $P(x), Q(x)$ は常に存在するとは限りません。存在する場合には，有理式 $\frac{P(x)}{Q(x)}$ は一意に定まります。このことは 3.3 節の解の一意性と同様に証明することができます。（ただし多項式の組 $(P(x), Q(x))$ としては，比を保ったまま取り換える自由度が残るため一意とは限りません。）

復元問題の最小位数を L とし， $2L \leq N$ が成り立つ場合には，復元問題の解 $(L, Q(x))$ から得られる有理式 $\frac{P(x)}{Q(x)}$ は， $S(x)$ の $(L-1, L)$ Padé 近似と解釈できます。

6.2 節のアルゴリズムは，Padé 近似を拡張 Euclid の互除法で求める方法とも対応します。

7. 関連問題

1. https://judge.yosupo.jp/problem/find_linear_recurrence
2. <https://yukicoder.me/problems/no/3228>
3. https://atcoder.jp/contests/tenka1-2015-qualb/tasks/tenka1_2015_qualB_c
4. <https://yukicoder.me/problems/no/2883>
5. <https://yukicoder.me/problems/no/3182>

6. <https://yukicoder.me/problems/no/3044>
7. https://atcoder.jp/contests/npcapc_2024/tasks/npcapc_2024_a
8. <https://yukicoder.me/problems/no/1516>
9. <https://yukicoder.me/problems/no/541>
10. <https://yukicoder.me/problems/no/579>
11. https://atcoder.jp/contests/arc182/tasks/arc182_c
12. https://atcoder.jp/contests/abc204/tasks/abc204_f
13. https://atcoder.jp/contests/tupc2023/tasks/tupc2023_n
14. https://atcoder.jp/contests/jscc2024-final/tasks/jscc2024_final_c

1 は本講座の復元問題そのものです。

その他の問題は基本的には BMBM 法で解ける問題ばかりです。特に 2, 3, 4, 5 あたりは先頭 N 項の計算が非常に簡単です。どのような位数の定数係数線形漸化式を持つのかを考えながら取り組んでください。

その他の問題も BMBM 法で解けるものばかりですが、行列を求めたりする必要はないものの、時には先頭 N 項の計算のために問題ごとに dp による解法を考える必要があります。それでも行列を具体的に書き下すよりは簡潔に実装できることが感じられると思います。

8. まとめ

本講座では、有限数列から最小位数の定数係数線形漸化式を復元する問題について解説しました。

復元問題を解く代表的なアルゴリズムとして、Berlekamp–Massey アルゴリズムを扱いました。Berlekamp–Massey アルゴリズムは、入力列を先頭から順に処理し、各時点での最小位数の漸化式を更新していくアルゴリズムです。

さらに、復元問題を Euclid の互除法によって解く方法も紹介しました。なお 6.5 節で補足したように、実はかなり近い関係にあります。実装の簡潔さという点では Berlekamp–Massey アルゴリズムの方が扱いやすいことが多いですが、Euclid の互除法による見方は、Half-GCD を用いた高速化にもつながります。

5 節では、競技プログラミングにおける典型的な利用方法として、BMBM 法を解説しました。これは、求めたい無限数列の先頭項を十分多く計算し、Berlekamp–Massey アルゴリズム

ムで線形漸化式を復元し、Bostan–Mori のアルゴリズムで第 K 項を求めるというものです。BMBM 法は考察や実装を簡略化できるだけでなく、計算量の改善につながることもあり、競技プログラミングにおいて非常に有用な手法です。

最後に、本講座では扱わなかった関連話題をいくつか挙げておきます。

ひとつは、係数環が体でない場合の線形漸化式の復元です。本講座では係数環を体として復元問題を扱いました。一方で、係数環が体ではない場合、例えば $\mathbb{Z}/m\mathbb{Z}$ で m が合成数である場合には、本講座のアルゴリズムをそのまま用いることはできません。 $\mathbb{Z}/m\mathbb{Z}$ の場合に復元問題を解くアルゴリズムとしては **Reeds–Sloane アルゴリズム** が知られており、以下の講座で扱います。

- 線形漸化式の復元 (mod m)

もうひとつの関連話題として、Black-box Linear Algebra との関係を紹介します。行列 M と行ベクトル u^T 、列ベクトル v に対して $A_i = u^T M^i v$ で定まる数列を考えます。もし M が

$$M^L = \sum_{j=1}^L c_j M^{L-j}$$

を満たすならば、数列 A は

$$A_i = \sum_{j=1}^L c_j A_{i-j} \quad (L \leq i)$$

を満たします。したがって、行列 M の最小多項式を求める問題と、数列 A_i の漸化式を復元する問題には密接な関係があります。実際、この関係と乱択アルゴリズムを組み合わせることで、 M の最小多項式を求められることが知られています。

このとき、行列 M の成分を直接扱う必要はなく、 M を列ベクトルにかけると操作だけが重要になります。このように、行列の成分を明示的に扱うのではなく、行列のベクトルへの作用だけを用いてその行列に関する情報を得る手法は **Black-box Linear Algebra** と呼ばれており、線形漸化式の復元はその際の基礎を支える道具となります。

9. 参考文献

1. Elwyn R. Berlekamp. Algebraic Coding Theory. McGraw-Hill. 1968.

2. James L. Massey. Shift-register synthesis and BCH decoding. IEEE Transactions on Information Theory. Vol. IT-15. No. 1. pp. 122–127. 1969. DOI: 10.1109/TIT.1969.1054260. <https://doi.org/10.1109/TIT.1969.1054260>.
3. Yasuo Sugiyama, Masao Kasahara, Shigeichi Hirasawa, Toshihiko Namekawa. A method for solving key equation for decoding Goppa codes. Information and Control. Vol. 27. No. 1. pp. 87–99. 1975. DOI: 10.1016/S0019-9958(75)90090-X. [https://doi.org/10.1016/S0019-9958\(75\)90090-X](https://doi.org/10.1016/S0019-9958(75)90090-X).
4. Jean Louis Dornstetter. On the Equivalence Between Berlekamp's and Euclid's Algorithms. IEEE Transactions on Information Theory. Vol. IT-33. No. 3. pp. 428–431. 1987. DOI: 10.1109/TIT.1987.1057299. <https://doi.org/10.1109/TIT.1987.1057299>.
5. Maria Bras-Amorós, Michael E. O'Sullivan. The Berlekamp-Massey Algorithm and the Euclidean Algorithm: a Closer Link. arXiv:0908.2198. 2009. <https://arxiv.org/abs/0908.2198>.
6. Wikipedia. Berlekamp–Masseyアルゴリズム. <https://ja.wikipedia.org/wiki/Berlekamp–Masseyアルゴリズム>. (閲覧日: 2026-06-17).
7. Wikipedia. パデ近似. <https://ja.wikipedia.org/wiki/パデ近似>. (閲覧日: 2026-06-17).
8. TLE. Linear Recurrence and Berlekamp-Massey Algorithm. Codeforces Blog. <https://codeforces.com/blog/entry/61306>. (閲覧日: 2026-06-17).
9. adamant. Recovering a linear recurrence with the extended Euclidean algorithm. Codeforces Blog. 2022. <https://codeforces.com/blog/entry/101628>. (閲覧日: 2026-06-17).
10. つちのこ. Berlekamp-MasseyやBostan-Moriを使うことで殴れる問題一覧. つちのこの日記. <https://tsuchi.hateblo.jp/entry/2021/10/09/124804>. (閲覧日: 2026-06-17).
11. sugarknri. Berlekamp–Massey algorithm. <https://sugarknri.hatenablog.com/entry/2017/11/18/234217>. (閲覧日: 2026-06-17).
12. OI Wiki. Berlekamp–Massey 算法. <https://oi-wiki.org/math/berlekamp-massey/>. (閲覧日: 2026-06-17).

トップページ：[AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など：[Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

[アクセスガイド](#)

[お知らせ](#)

資料請求

[AtCoder](#)

[AtCoderJobs](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[AtCoderCareerDesign](#)

AtCoder株式会社について

[会社概要](#)

[FAQ](#)

[プライバシーポリシー](#)

